Building Agents: The Four Engineering Layers

Scaler School of Technology | ML System Design and LLMOps | Week 4 — Bonus Session | Post-Lecture Notes

The Four-Layer Cheat Sheet

The agent quality ladder has four rungs. Each fixes a class of failures; they compose.

Layer	Fixes	Key Moves
Prompt	Vague/malformed output, hallucinated specifics	Role & goal; JSON output contract; step-by-step reasoning; "cite a line"; one-shot exemplar
Context	Missing info, blown budget, cost spiral	Retrieve, don't dump; compact summaries; order (relevant last); curate tool results; scratchpad
Loop	Can't do multi-step tasks, runs forever, oscillates	Explicit stop signal; hard step cap; repeated-call detection; re-plan on stall; carry state forward
Harness	Unsafe actions, confidently-wrong output, cost chaos	Read-only sandbox; arg validation; verification gate; cost ceiling; human-in-the-loop; full trace

Symptom → Layer: Diagnosis Guide

Prompt Layer

- Vague or unstructured output
- Invents specifics (hallucination)
- Ignores output format rules

Context Layer

- Missing required information
- Model costs spiral upward
- Latency creeps up

Loop Layer

- Can't solve multi-step problems
- Loops forever or oscillates
- Repeated calls to the same tool

Harness Layer

- Unsafe/irreversible actions taken
- Confidently wrong answers
- Cost or behavior unpredictable

Principle: They compose. They don't substitute.

What the Practical Showed

Stage	F1	$/run	steps	latency (s)
0 – naive	0.00	0.00022	1.0	6
1 – prompt + context	0.75	0.00015	1.0	5
2 – loop + harness	0.75	0.00036	2.8	16

Prompt + context moved F1 from 0.00 to 0.75 for almost no extra cost. A JSON output contract, a role statement, and "cite a specific line or don't report it" fix the parseability problem; retrieving the enclosing code and the out-of-diff callers fixes the missing-information problem. They land in the same call, so their payoff shows up together.

Loop + harness did *not* move F1 further on this slice — the model was already sharp enough to find these particular bugs by reasoning statically. What moved instead: steps climbed from 1.0 to 2.8 and latency from 5s to 16s, because the agent now *verifies* before it answers — it calls `run_tests` to confirm a suspicion instead of asserting it, and every finding is re-grounded against the actual diff before it ships (precision 0.75). Not every layer has to move F1; some pay off in confidence and predictability instead. The human-in-the-loop flag catches any finding that looks edge-case or rare.

Token Economics Worked Example

A 10-turn agent session, input tokens only

A typical review agent replays the system prompt, the PR metadata, findings so far, and conversation history at each turn. By turn 10, that's roughly 4,000 input tokens/turn. Over 10 turns: $4{,}000 \times 10 = 40{,}000$ input tokens per run. At 1,000 runs per day, that's **40M input tokens/day** from a single deployed agent.

Context engineering cuts the per-turn budget (compact summaries, summaries of summaries). Prompt caching means the stable system prefix is cached; only the latest finding is sent fresh each turn. Upshot: massive TCO reduction with no loss of quality.

The Loop + Harness Stage: Annotated Pseudocode

```
findings = []
seen_calls = set()
for step in range(MAX_STEPS):                # loop: hard cap
    if trace.cost() > COST_CEILING:          # harness: cost limit
        break

    resp = model(system, messages, tools)  # context: cached prefix

    if resp.tool_calls:
        for call in resp.tool_calls:
            if call.name not in READONLY_TOOLS:    # harness: sandbox
                observe("REJECTED")
                continue

            try:
                validate_args(call)          # harness: schema check
            except ValueError:
                observe("INVALID"); continue

            sig = (call.name, frozenset(call.args.items()))
            if sig in seen_calls:            # loop: suppress repeats
                observe("SKIPPED"); continue
            seen_calls.add(sig)

            result = dispatch(call)
            observe(result)
        continue

    # Reached a final answer
    raw = parse_json(resp.text)              # prompt: struct contract
    for finding in raw:
        grounded = verify_against_diff(finding)  # harness: re-ground
        if is_rare_or_edge_case(grounded):
            flag_for_human(grounded)         # harness: HitL
        else:
            findings.append(grounded)
```

```
    return findings
```

Key annotations: Loop prevents infinite recursion and bounds cost. Harness defends against unsafe calls and grounds findings in reality. Context uses a cached system prompt. Prompt enforces JSON structure.

Exercises

Work through these independently. Code sketches welcome; full implementations not required.

Exercise 1: Add a Tool Safely

Write a `list_symbols(file)` tool that returns the names and line numbers of all top-level functions and classes in a Python file. (a) Specify the JSON schema (name, description, parameters, required, strict). (b) Add it to the read-only allow-list in the harness. (c) Write a single test that confirms a `write_cache()` call is still rejected by the sandbox, but your new tool is accepted.

Exercise 2: Detect Oscillation

The current loop detects identical repeated calls. Extend it to detect *oscillation*: if the last four tool calls follow an A, B, A, B pattern (or any 2-cycle), force a final answer without waiting for the model to stop. Pseudocode: track the last 4 calls; on each new call, check if the 4-call window forms a 2-cycle; if yes, set `step = MAX_STEPS - 1` to trigger the final-answer path on the next iteration.

Exercise 3: Extend the Evaluation Fixture

The fixture has 12 PRs covering 10 bug classes. Add a 13th PR with a planted bug *not* already in the fixture (e.g. a silent data loss via mutable default argument, or an incorrect type coercion). Add its label to `labels.json`, re-run the recording script, and check which iteration first catches it. Did harness add value for this bug, or was loop enough?

Further Reading

- **Anthropic — "Building effective agents"** — Core patterns: ReAct, tool use, memory. Essential primer before shipping an agent.
- **Anthropic — Engineering posts on context engineering and prompt caching** — How to cut token spend by 50–80% on long-running agents.
- **Yao et al. (2022) — "ReAct: Synergizing Reasoning and Acting in Language Models"** — arXiv:2210.03629. The loop pattern we use is directly inspired by ReAct.
- **Shinn et al. (2023) — "Reflexion: Language Agents with Verbal Reinforcement Learning"** — How agents can self-correct and learn from failure. Appendix C has production pitfalls.
- **Liu et al. (2023) — "Lost in the Middle: How Language Models Use Long Contexts"** — Critical reading: why order of context matters, and why a 50k-token prompt can perform worse than a 10k one.
- **Langfuse Docs — Tracing & Sessions** — How to instrument a Python agent for observability. Covers cost tracking, latency, tool-call traces, and session replay.